توابع و نحوه سازمان‌دهی کد در پایتون

وقتی برنامه‌ای می‌نویسیم، ممکن است یک عملیات در چند قسمت از برنامه موردنیاز باشد یا تعداد دستورها به‌مرور زیاد شود. اگر همه کدها را پشت سر هم بنویسیم، درک برنامه، پیدا کردن خطاها و ایجاد تغییرات دشوارتر می‌شود.

برای حل این مسئله، از توابع (Functions) استفاده می‌کنیم. تابع بخشی از کد است که وظیفه مشخصی دارد و می‌توانیم هر زمان لازم بود آن را اجرا کنیم.

توابع به ما کمک می‌کنند کدهای تکراری را کاهش دهیم، وظایف مختلف را از یکدیگر جدا کنیم و برنامه‌هایی خواناتر و قابل-نگهداری‌تر بسازیم.

۱. تابع چیست؟

فرض کنید می‌خواهیم چند بار پیام خوشامدگویی نمایش دهیم. می‌توانیم دستور `print()` را چند بار تکرار کنیم؛ اما اگر این عملیات در قسمت‌های مختلف برنامه موردنیاز باشد، بهتر است آن را در قالب یک تابع تعریف کنیم.

برای تعریف تابع در پایتون، از کلمه کلیدی `def` استفاده می‌کنیم:

```
def greet():
    print("Hello!")
    print("Welcome to Python.")


greet()
```

خروجی:

```
Hello!
Welcome to Python.
```

در این مثال:

- `def` برای تعریف تابع استفاده می‌شود.
- `greet` نام تابع است.
- پرانتزها محل مشخص‌کردن ورودی‌های تابع هستند.
- دستورهای داخل تابع با تورفتگی نوشته می‌شوند.
- عبارت `greet()` تابع را اجرا یا به‌اصطلاح فراخوانی می‌کند.

نکته مهم: تعریف تابع به‌تنهایی باعث اجرای دستورهای داخل آن نمی‌شود. برای اجرای تابع باید آن را فراخوانی کنیم.

۲. چرا از تابع استفاده می‌کنیم؟

فرض کنید می‌خواهیم مبلغ نهایی یک خرید را با احتساب مالیات محاسبه کنیم. اگر این محاسبه در بخش‌های مختلف برنامه تکرار شود، ممکن است چند بار همان کد را بنویسیم.

با استفاده از تابع، می‌توانیم محاسبه را یک بار تعریف کنیم و هر جا لازم بود از آن استفاده کنیم:

```
def calculate_total(price, tax):
    total = price + tax
    print(total)


calculate_total(100, 10)
calculate_total(250, 25)
calculate_total(80, 8)
```

خروجی:

```
110
275
88
```

در این مثال، تابع `calculate_total()` محاسبه را انجام می‌دهد و هر بار با مقادیر متفاوتی فراخوانی می‌شود.

مزیت این روش آن است که اگر بخواهیم شیوه محاسبه را تغییر دهیم، معمولاً کافی است فقط یک قسمت از کد را اصلاح کنیم.

۳. پارامترها و آرگومان‌ها

بسیاری از توابع برای انجام وظیفه خود به اطلاعاتی نیاز دارند. این اطلاعات از طریق پارامترها و آرگومان‌ها دریافت می‌شوند.

در مثال قبلی:

```
def calculate_total(price, tax):
    print(price + tax)


calculate_total(100, 10)
```

متغیرهای `price` و `tax` در تعریف تابع، پارامتر نامیده می‌شوند. مقادیر 100 و 10 که هنگام فراخوانی به تابع داده‌ایم، آرگومان هستند.

پارامترها به ما اجازه می‌دهند یک تابع را برای داده‌های مختلف به کار ببریم.

برای مثال، می‌توانیم تابعی بنویسیم که نام فرد را دریافت کند و پیام مناسبی نمایش دهد:

```
def greet_user(name):
    print(f"Hello, {name}!")


greet_user("Sara")
greet_user("Ali")
```

خروجی:

```
Hello, Sara!
Hello, Ali!
```

تابع برای هر ورودی، پیام متناسب با همان مقدار را تولید می‌کند.

۴. مقدار بازگشتی و دستور `return`

گاهی هدف تابع فقط نمایش نتیجه نیست؛ بلکه می‌خواهیم نتیجه محاسبه را در اختیار بخش دیگری از برنامه قرار دهیم.

برای این کار از دستور `return` استفاده می‌کنیم.

```
def calculate_total(price, tax):
    total = price + tax
    return total


result = calculate_total(100, 10)

print(result)
print(result * 2)
```

خروجی:

```
110
220
```

در این مثال، تابع نتیجه محاسبه را با `return` برمی‌گرداند. سپس آن مقدار در متغیر `result` ذخیره می‌شود و می‌توانیم از آن در محاسبات دیگر نیز استفاده کنیم.

تفاوت `return` و `print()`

این دو دستور کاربرد یکسانی ندارند:

- `print()` مقدار را روی صفحه نمایش می‌دهد.
- `return` مقدار را از تابع به محل فراخوانی برمی‌گرداند و اجرای همان فراخوانی تابع را پایان می‌دهد.

برای روشن‌تر شدن تفاوت، دو تابع زیر را در نظر بگیرید:

```
def add_and_print(a, b):
    print(a + b)


def add_and_return(a, b):
    return a + b


result_1 = add_and_print(3, 4)
result_2 = add_and_return(3, 4)

print("First result:", result_1)
print("Second result:", result_2)
```

خروجی:

```
7
First result: None
Second result: 7
```

تابع اول فقط نتیجه را نمایش می‌دهد. چون مقدار مشخصی را با `return` برنمی‌گرداند، نتیجه فراخوانی آن `None` است.

تابع دوم مقدار حاصل از جمع را برمی‌گرداند؛ بنابراین می‌توانیم آن را در متغیر ذخیره کنیم یا در محاسبات دیگری به کار ببریم.

در بسیاری از توابعی که قرار است بخشی از یک برنامه بزرگ‌تر باشند، برگرداندن نتیجه با `return` کاربردی‌تر است.

۵. پارامترهای پیش‌فرض

گاهی می‌خواهیم یک پارامتر در حالت عادی مقدار مشخصی داشته باشد، اما در صورت نیاز بتوانیم مقدار دیگری برای آن تعیین کنیم.

برای این کار از پارامتر پیش‌فرض استفاده می‌کنیم:

```
def greet_user(name, message="Welcome!"):
    print(f"{message} {name}")


greet_user("Sara")
greet_user("Ali", "Good morning!")
```

خروجی:

```
Welcome! Sara
Good morning! Ali
```

در فراخوانی اول، فقط نام داده شده است؛ بنابراین مقدار پیش‌فرض `message` استفاده می‌شود. در فراخوانی دوم، پیام موردنظر را نیز مشخص کرده‌ایم.

پارامترهای پیش‌فرض به ما کمک می‌کنند بدون پیچیده‌کردن تابع، حالت‌های رایج را پوشش دهیم.

۶. توابع می‌توانند از توابع دیگر استفاده کنند

در یک برنامه، لازم نیست همه وظایف را داخل یک تابع انجام دهیم. می‌توانیم وظایف را به چند تابع تقسیم کنیم و از آن‌ها در کنار هم استفاده کنیم.

برای مثال، فرض کنید می‌خواهیم مبلغ نهایی خرید را پس از اعمال تخفیف محاسبه کنیم:

```
def calculate_discount(price, discount_percent):
    return price * discount_percent / 100


def calculate_final_price(price, discount_percent):
    discount = calculate_discount(price, discount_percent)
    return price - discount


final_price = calculate_final_price(200, 10)

print(final_price)
```

خروجی:

```
180.0
```

در این مثال، هر تابع وظیفه مشخصی دارد:

- `calculate_discount()` مقدار تخفیف را محاسبه می‌کند.
- `calculate_final_price()` از تابع اول استفاده می‌کند و قیمت نهایی را به دست می‌آورد.

اگر در ادامه بخواهیم روش محاسبه تخفیف را تغییر دهیم، می‌توانیم تابع مربوط به آن را جداگانه بررسی کنیم.

اصل مهم: بهتر است هر تابع یک وظیفه روشن داشته باشد. تابعی که کارهای نامرتبط زیادی انجام می‌دهد، معمولاً سخت‌تر فهمیده می‌شود و تغییر آن نیز دشوارتر است.

۷. متغیرهای داخل تابع

متغیرهایی که داخل یک تابع تعریف می‌شوند، معمولاً محلی هستند؛ یعنی خارج از همان تابع مستقیماً در دسترس نیستند.

برای مثال:

```
def calculate_total():
    price = 100
    tax = 10
    return price + tax


print(calculate_total())
```

خروجی:

```
110
```

در این مثال، متغیرهای `price` و `tax` در داخل تابع تعریف شده‌اند و برای انجام محاسبه همان تابع به کار می‌روند.

اما اگر بخواهیم بیرون از تابع به متغیری دسترسی داشته باشیم، باید به محدوده تعریف آن توجه کنیم:

```
def calculate_total():
    price = 100
    tax = 10
    return price + tax


print(calculate_total())
print(price)  # This raises a NameError
```

دستور آخر خطا ایجاد می‌کند؛ زیرا متغیر price فقط داخل تابع تعریف شده و در محدوده بیرونی شناخته‌شده نیست.

این مفهوم را محدوده متغیر (Scope) می‌نامیم.

در شروع کار، بهتر است داده‌های موردنیاز تابع را از طریق پارامترها دریافت کنیم و نتایج را با return برگردانیم. این روش وابستگی میان قسمت‌های مختلف برنامه را کمتر می‌کند.

۸. سازمان‌دهی کد با استفاده از توابع

اکنون می‌توانیم از توابع برای سازمان‌دهی یک برنامه کوچک استفاده کنیم.

فرض کنید می‌خواهیم مبلغ خرید را دریافت کنیم، تخفیف را محاسبه کنیم و نتیجه را نمایش دهیم. به‌جای قرار دادن همه دستورها در یک قسمت، هر وظیفه را در تابع جداگانه‌ای قرار می‌دهیم:

```
def calculate_discount(price):
    if price >= 100:
        return price * 0.10

    return 0


def calculate_final_price(price):
    discount = calculate_discount(price)
    return price - discount


def main():
    price = float(input("Enter the price: "))

    if price < 0:
        print("Invalid price.")
        return

    final_price = calculate_final_price(price)

    print(f"Final price: {final_price}")


if __name__ == "__main__":
    main()
```

این برنامه را می‌توان با وارد کردن یک قیمت مانند 150 اجرا کرد. چون مبلغ خرید حداقل ۱۰۰ است، تخفیف ۱۰ درصدی در نظر گرفته می‌شود و قیمت نهایی نمایش داده خواهد شد.

در این برنامه چند نکته مهم وجود دارد.

اول، هر تابع مسئولیت مشخصی دارد.

- calculate_discount() تخفیف را محاسبه می‌کند.

- `calculate_final_price()` قیمت نهایی را به دست می‌آورد.
- `main()` جریان اصلی اجرای برنامه را مدیریت می‌کند.

دوم، توابع از طریق ورودی و خروجی با یکدیگر ارتباط دارند. به‌جای آنکه هر تابع همه جزئیات را بداند، داده‌های موردنیاز خود را دریافت می‌کند و نتیجه را برمی‌گرداند.

سوم، بخش پایانی برنامه مشخص می‌کند که تابع اصلی چه زمانی اجرا شود:

```
if __name__ == "__main__":
    main()
```

این الگو به‌ویژه زمانی مفید است که بخواهیم یک فایل پایتون هم مستقیماً اجرا شود و هم بتوانیم توابع آن را از فایل دیگری وارد کنیم.

در برنامه‌های بسیار کوچک، استفاده از این ساختار همیشه ضروری نیست؛ اما با بزرگ‌تر شدن برنامه، به منظم نگه‌داشتن کد کمک می‌کند.

نکته دیگر این است که این برنامه ورودی‌های منفی را بررسی می‌کند، اما هنوز ورودی غیرعددی را مدیریت نمی‌کند. برای مثال، واردکردن متن به‌جای قیمت باعث خطا در تبدیل با `float()` می‌شود. مدیریت این وضعیت را می‌توانیم در مبحث خطاها بررسی کنیم.

۹. چه زمانی باید کد را به تابع تبدیل کنیم؟

هر دستور یا عبارت کوتاهی به تابع جداگانه نیاز ندارد. هدف، سازمان‌دهی بهتر برنامه است، نه زیاد کردن تعداد توابع.

معمولاً تعریف تابع زمانی مفید است که:

- یک عملیات در چند قسمت برنامه تکرار می‌شود.
- می‌توان وظیفه‌ای مشخص را از بقیه کد جدا کرد.
- یک بخش از برنامه به‌اندازه‌ای طولانی شده که درک آن دشوار است.
- می‌خواهیم یک قسمت را مستقل از بقیه برنامه آزمایش کنیم.
- احتمال می‌دهیم آن عملیات در آینده تغییر کند یا دوباره استفاده شود.

برای مثال، اگر قرار است محاسبه مالیات در چند قسمت برنامه استفاده شود، بهتر است آن را در تابعی مستقل قرار دهیم. به این ترتیب، تغییر نرخ مالیات یا روش محاسبه ساده‌تر خواهد بود.

در مقابل، اگر یک دستور ساده فقط یک بار استفاده می‌شود و جداکردن آن هیچ کمکی به فهم کد نمی‌کند، لزوماً نیازی به تبدیل آن به تابع نیست.

جمع‌بندی

تابع بخشی از کد است که می‌توان آن را تعریف کرد و هر زمان لازم بود فراخوانی کرد. با استفاده از پارامترها، داده‌های موردنیاز را به تابع می‌دهیم و با `return` می‌توانیم نتیجه را به بخش دیگری از برنامه برگردانیم.

توابع به کاهش کدهای تکراری، تقسیم مسئله به بخش‌های کوچک‌تر و افزایش خوانایی و قابلیت نگهداری برنامه کمک می‌کنند.

در عصر هوش مصنوعی، شناخت توابع به ما امکان می‌دهد کدهای تولیدشده را بهتر بررسی کنیم و در صورت نیاز، ساختار آن‌ها را اصلاح کنیم.

تمرین

برنامه‌ای بنویسید که هزینه خرید چند محصول را محاسبه کند و برای این کار از چند تابع استفاده کند.

برنامه باید:

1. تابعی به نام `calculate_total()` داشته باشد که قیمت و تعداد یک محصول را دریافت کند و هزینه آن را برگرداند.
2. تابعی به نام `calculate_discount()` داشته باشد که مبلغ خرید را دریافت کند؛ اگر مبلغ خرید حداقل ۵۰۰ واحد بود، ۱۰ درصد تخفیف محاسبه کند و در غیر این صورت صفر برگرداند.
3. تابعی به نام `main()` داشته باشد که اطلاعات دو محصول را دریافت کند، هزینه آن‌ها را محاسبه کند و پس از اعمال تخفیف، مبلغ نهایی را نمایش دهد.

تمرین تکمیلی: از هوش مصنوعی بخواهید کد خود را بررسی کند و پیشنهاد دهد چگونه می‌توان خوانایی و سازمان‌دهی آن را بهتر کرد. سپس پیشنهادها را بخوانید و تصمیم بگیرید کدام‌یک واقعاً مفید هستند.